

17. 上位探索ソルバ

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	5
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	7
6.6 list、dict、deque、NumPy 配列、pandas DataFrame	7
6.7 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	7
6.8 Cross-Entropy Method を式と実装で理解する	7
6.9 warm start は何を保存し、どう効くか	8
6.10 freshness、filter、guard、fallback、fail-safe	8
6.11 制御、状態、入力、モデル予測制御 MPC	8
6.12 単体試験、SILS、replay、観測可能性	9
7 関数・クラスを上から順に解説	9
7.1 L19 関数 _fork_finite_cost	9

7.2	L25 関数 clip_to_bounds	10
7.3	L32 関数 finite_cost	11
7.4	L42 関数 default_seed_library	12
7.5	L57 関数 hybrid_bounded_minimize	13
7.6	L91 関数 hybrid_bounded_minimize.add_candidate	17
7.7	L143 関数 hybrid_bounded_minimize.local_refine_rows	18
7.8	L201 関数 hybrid_bounded_minimize.emit_progress	19
7.9	L369 関数 hybrid_bounded_minimize.should_run_cem_from_seed_consensus	20
8	処理の流れ	21
9	このファイルの位置づけ	22

1 対象

項目	内容
ファイル	mpc_solarcar/upper_solver.py
ソース SHA-256	0ea0437c48c25f839a8edbfff7e74f8d613a11451728eb7751db21756dd76be2a
種別	Python
区分	planner helper
役割	bounded global candidate search、CEM、SHGO、L-BFGS-B を束ねて upper policy を最適化する。
起動文脈	upper planner の数値最適化 backend。

2 このファイルを読む理由

bounded global candidate search、CEM、SHGO、L-BFGS-B を束ねて upper policy を最適化する。

- hybrid_bounded_minimize が中心。
- global 探索と local refine の接着層である。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_node.py
- scripts/solar_sim.py

3.2 次に読むと理解が進むファイル

- 特記事項なし。

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
_FORK_COST_FN: Callable[[np.ndarray], float] | None = None

def _fork_finite_cost(vec: np.ndarray) -> float:
    if _FORK_COST_FN is None:
        return float("inf")
    return finite_cost(_FORK_COST_FN, vec)
```

```

def clip_to_bounds(vec: np.ndarray, bounds: Bounds) -> np.ndarray:
    out = np.asarray(vec, dtype=float).copy()
    for idx, (lo, hi) in enumerate(bounds):
        out[idx] = float(np.clip(out[idx], lo, hi))
    return out

def finite_cost(cost_fn: Callable[[np.ndarray], float], vec: np.ndarray) -> float:
    try:
        value = float(cost_fn(np.asarray(vec, dtype=float)))
    except Exception:
        return float("inf")
    if not np.isfinite(value):

```

5 主要構造の読み取り

主要関数は clip_to_bounds, finite_cost, default_seed_library, hybrid_bounded_minimize, add_candidate, local_refine_rows, emit_progress, record_grid_candidate。

5.1 主要クラス・関数

行	種別	名前	補足
19	function	_fork_finite_cost	args: vec
25	function	clip_to_bounds	args: vec, bounds
32	function	finite_cost	args: cost_fn, vec
42	function	default_seed_library	args: x0, bounds
57	function	hybrid_bounded_minimize	args: cost_fn, x0, bounds
91	function	add_candidate	args: label, vec
143	function	local_refine_rows	args: pool
201	function	emit_progress	args: payload
225	function	record_grid_candidate	args: idx, candidate_x, candidate_fun
369	function	should_run_cem_from_seed_consensus	args: rows

5.2 ファイルを上から読んだときの定義順

行	内容
13	Bounds に Sequence[Tuple[float, float]] の結果を代入する。
16	_FORK_COST_FN に None を代入する。
19	関数 _fork_finite_cost を定義する。
25	関数 clip_to_bounds を定義する。
32	関数 finite_cost を定義する。
42	関数 default_seed_library を定義する。
57	関数 hybrid_bounded_minimize を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future__ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>from concurrent. futures import ProcessPoolExecutor, ThreadPoolExecutor</code>	<code>concurrent.futures</code> から <code>ProcessPoolExecutor</code> , <code>ThreadPoolExecutor</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L273, L280。
4	<code>from itertools import product</code>	<code>itertools</code> から <code>product</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L260。
5	<code>import multiprocessing as mp</code>	<code>multiprocessing</code> モジュールを利用するため。このファイル内での主な使用位置は L275。
6	<code>import os</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L270。
7	<code>from typing import Callable, Iterable, List, Sequence, Tuple</code>	<code>typing</code> から <code>Callable</code> , <code>Iterable</code> , <code>List</code> , <code>Sequence</code> , <code>Tuple</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L13, L16, L32, L42, L47, L58, L63, L75, ...。
9	<code>import numpy as np</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L16, L19, L25, L26, L28, L32, L34, L37, ...。
10	<code>from scipy. optimize import minimize, shgo</code>	<code>scipy.optimize</code> から <code>minimize</code> , <code>shgo</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L148, L311。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()  
alias = node  
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や ‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’ となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.7 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)
- [SciPy 公式: scipy.optimize.shgo](#)

6.8 Cross-Entropy Method を式と実装で理解する

CEM は候補を生成する確率分布を持ち、良かった elite 候補から分布を更新する反復的な確率最適化である。このリポジトリの upper_solver.py は各制御点速度を独立正規分布で生成し、上下限へ clip する。

$$u_i^{(j)} \sim \mathcal{N}(\mu_i^{(g)}, (\sigma_i^{(g)})^2), \quad u_i^{(j)} \leftarrow \text{clip}(u_i^{(j)}, l_i, h_i)$$

$$\mu_i^{(g+1)} = \frac{1}{K} \sum_{j \in \mathcal{E}_g} u_i^{(j)}, \quad \sigma_i^{(g+1)} = \max \left(\text{Std}_{j \in \mathcal{E}_g} u_i^{(j)}, 0.05(h_i - l_i) \right)$$

ここで E_g は cost が小さい上位 K 候補である。平均は良い領域へ移り、標準偏差は探索幅を表す。標準偏差の下限は探索が完全に潰れることを避ける。

現行 `hybrid_bounded_minimize` は、`deterministic seed` を評価し、上位候補を `L-BFGS-B` で局所 refine し、設定と `seed` 間不一致に応じて CEM を実行し、最後に再度局所 refine する。したがって CEM 単独ではなく `hybrid solver` である。

CEM で落とした候補を永久保存しないこと自体は通常の最適化として自然だが、`off-nominal` 状態からの再利用には別の `policy library` 設計が必要である。状態を無制限に全組合せ保存する代わりに、`SoC`、進捗、時刻、予報誤差、停止状態などの `scenario` を設計し、近傍 `policy` を検索して MPC で再最適化する。

根拠資料

- [Rubinstein and Kroese: The Cross-Entropy Method](#)
- [SciPy 公式: `scipy.optimize.minimize`](#)

6.9 warm start は何を保存し、どう効くか

warm start は前回または offline 探索で得た入力系列を、次の最適化の初期候補として渡すことである。答えを固定するのではなく、探索開始点と候補 `library` の一員を与える。

$$u_i^{0,new} = \text{interp} \left(s_i^{new}; s_j^{old}, u_j^{*,old} \right)$$

距離基準計画では、現在より後ろの旧制御点を捨て、新しい絶対距離制御点へ補間して shift する。初期状態や天候が変われば cost 評価は新条件で行われるので、warm start が不適切でも他 `seed`、CEM、安全 fallback が補う設計が必要である。

warm start の効き方は、局所法なら収束先と反復数、CEM なら初期 mean または候補 `pool`、`receding horizon` なら前回解の時間・距離 shift として現れる。どの位置に渡しているかを呼出引数まで追う。

根拠資料

- [SciPy 公式: `scipy.optimize.minimize`](#)
- [Rubinstein and Kroese: The Cross-Entropy Method](#)

6.10 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と `timeout` から `freshness` を判定し、`stale` 値を計画状態へ無条件に同期しない。

`filter` は `noise` と一時的な飛び値を抑えるが、遅れを生む。`slew limit` は指令変化率を制限する。安全 `guard` は `solver` の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

`fallback` は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 `mode` ごとに選ぶ。`fallback` 発生は `status` と `log` へ残し、正常解と区別する。

6.11 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは `SoC`、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.12 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- ROS 2 Humble 公式: [rqt_graph](#)
- ROS 2 Humble 公式: [Recording and playing back data](#)
- ROS 2 公式: [Executors](#)

7 関数・クラスを上から順に解説

7.1 L19 関数 `_fork_finite_cost`

- 行範囲: L19–L22
- 所属: module 直下
- 定義: `_fork_finite_cost(vec:np.ndarray)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `finite_cost, float`
- 戻り値の要点: `finite_cost(_FORK_COST_FN, vec) / float('inf')`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `_FORK_COST_FN is None` を判定し、真なら内部処理を行う。
2. `float('inf')` を返す。
3. `finite_cost(_FORK_COST_FN, vec)` を返す。

代表コード断片

```
def _fork_finite_cost(vec: np.ndarray) -> float:
    if _FORK_COST_FN is None:
        return float("inf")
    return finite_cost(_FORK_COST_FN, vec)
```

7.2 L25 関数 `clip_to_bounds`

- 行範囲: L25–L29
- 所属: module 直下
- 定義: `clip_to_bounds(vec: np.ndarray, bounds: Bounds) -> np.ndarray`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `clip`, `copy`, `enumerate`, `float`
- 戻り値の要点: `out`
- 主なローカル代入名: `hi`, `idx`, `lo`, `out`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `out` に `np.asarray(vec, dtype=float).copy()` の結果を代入する。
2. `enumerate(bounds)` を順に走査し、各要素を `(idx, (lo, hi))` に入れて処理する。
3. `out[idx]` に `float(np.clip(out[idx], lo, hi))` の結果を代入する。
4. `out` を返す。

代表コード断片

```
def clip_to_bounds(vec: np.ndarray, bounds: Bounds) -> np.ndarray:
    out = np.asarray(vec, dtype=float).copy()
    for idx, (lo, hi) in enumerate(bounds):
        out[idx] = float(np.clip(out[idx], lo, hi))
    return out
```

7.3 L32 関数 `finite_cost`

- 行範囲: L32–L39
- 所属: module 直下
- 定義: `finite_cost(cost_fn:Callable[[np.ndarray],float],vec:np.ndarray)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `cost_fn`, `float`, `isfinite`
- 戻り値の要点: `value / float('inf') / float('inf')`
- 主なローカル代入名: `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. `value` に `float(cost_fn(np.asarray(vec, dtype=float)))` の結果を代入する。
3. Exception を捕捉した場合:
4. `float('inf')` を返す。
5. 条件 `not np.isfinite(value)` を判定し、真なら内部処理を行う。
6. `float('inf')` を返す。
7. `value` を返す。

代表コード断片

```
def finite_cost(cost_fn: Callable[[np.ndarray], float], vec: np.ndarray) -> float:
    try:
        value = float(cost_fn(np.asarray(vec, dtype=float)))
    except Exception:
        return float("inf")
    if not np.isfinite(value):
        return float("inf")
    return value
```

7.4 L42 関数 `default_seed_library`

- 行範囲: L42–L54
- 所属: module 直下
- 定義: `default_seed_library(x0:np.ndarray, bounds:Bounds)->List[tuple[str,np.ndarray]]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `append`, `array`, `asarray`, `clip_to_bounds`, `float`, `len`, `linspace`, `maximum`
- 戻り値の要点: `seeds`
- 主なローカル代入名: `_`, `const`, `frac`, `hi`, `hi_i`, `lo`, `lo_i`, `seeds`, `span`, `x0`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、try 0

この定義を読むための Python 構文

- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `x0` に `np.asarray(x0, dtype=float)` の結果を代入する。
2. `lo` に `np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)` の結果を代入する。
3. `hi` に `np.array([float(hi_i) for _, hi_i in bounds], dtype=float)` の結果を代入する。
4. `span` に `np.maximum(hi - lo, 1e-06)` の結果を代入する。
5. `seeds` に `[('warm_start', clip_to_bounds(x0, bounds))]` を代入する。
6. (0.25, 0.4, 0.55, 0.7, 0.85) を順に走査し、各要素を `frac` に入れて処理する。
7. `const` に `lo + frac * span` の結果を代入する。
8. `seeds.append(...)` を実行する。
9. `seeds.append(...)` を実行する。
10. `seeds.append(...)` を実行する。
11. `seeds.append(...)` を実行する。
12. `seeds` を返す。

代表コード断片

```
def default_seed_library(x0: np.ndarray, bounds: Bounds) -> List[tuple[str, np.ndarray]]:
    x0 = np.asarray(x0, dtype=float)
    lo = np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)
    hi = np.array([float(hi_i) for _, hi_i in bounds], dtype=float)
    span = np.maximum(hi - lo, 1.0e-6)
    seeds: List[tuple[str, np.ndarray]] = [("warm_start", clip_to_bounds(x0, bounds))]
    for frac in (0.25, 0.40, 0.55, 0.70, 0.85):
        const = lo + frac * span
        seeds.append((f"const_{frac:.2f}", clip_to_bounds(const, bounds)))
        seeds.append((f"warm_mix_{frac:.2f}", clip_to_bounds(0.5 * x0 + 0.5 * const, bounds)))
    seeds.append(("ramp_up", clip_to_bounds(np.linspace(lo[0], hi[0], len(x0)), bounds)))
    seeds.append(("ramp_down", clip_to_bounds(np.linspace(hi[0], lo[0], len(x0)), bounds)))
    return seeds
```

7.5 L57 関数 hybrid_bounded_minimize

- 行範囲: L57–L473
- 所属: module 直下
- 定義: `hybrid_bounded_minimize(cost_fn: Callable[[np.ndarray], float], x0: np.ndarray, bounds: Bounds, *, maxiter: int, structured_seeds: Iterable[tuple[str, np.ndarray]] | None = None, cem_enabled: bool = True, cem_mode: str = 'auto', cem_generations: int = 4, cem_population: int = 16, cem_elite: int = 4, local_refine_topk: int = 4, seed_library_mode: str = 'full', rng_seed: int = 0, shgo_samples: int = 256, shgo_iters: int = 2, cert_grid_levels: int = 0, cert_grid_values: Sequence[float] | None = None, cert_max_evaluations: int = 250000, cert_workers: int = 1, progress_callback: Callable[[dict], None] | None = None, cert_progress_interval: int = 25) -> tuple[np.ndarray, dict]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ProcessPoolExecutor`, `ThreadPoolExecutor`, `abs`, `add`, `add_candidate`, `all`, `append`, `array`, `asarray`, `bool`, `clip`, `clip_to_bounds`
- 戻り値の要点: `(np.asarray(best['x'], dtype=float), {'success': bool(best.get('success', True)), 'fun': float(best['fun']), 'label': str(best.get('label', 'best')), 'nit': int(best.get('nit', 0) or 0), 'method': str(best.get('source', 'seed')), 'message': str(best.get('message', '')), 'candidates_evaluated': int(len(evaluated) + discrete_grid_candidates + shgo_nfev), 'cem_mode': cem_mode_norm, 'cem_used': cem_used, 'seed_consensus_spread': consensus_spread, 'shgo_used': shgo_used, 'shgo_nfev': shgo_nfev, 'shgo_local_minima': shgo_local_minima, 'discrete_global_proof': discrete_global_proof, 'discrete_grid_levels': discrete_grid_levels, 'discrete_grid_values': grid_values.tolist() if grid_values.size else [], 'discrete_grid_candidates': discrete_grid_candidates, 'discrete_grid_nonfinite': discrete_grid_nonfinite, 'discrete_grid_best_fun': discrete_grid_best_fun, 'discrete_grid_best_x': discrete_grid_best_x, 'deterministic_seed_candidates': deterministic_seed_candidates, 'deterministic_seed_nonfinite': deterministic_seed_nonfinite, 'finite_library_candidates': int(deterministic_seed_candidates + discrete_grid_candidates), 'seed_library_mode': seed_library_mode_norm, 'selected_x': np.asarray(best['x'], dtype=float).tolist(), 'selected_no_worse_than_grid': selected_no_worse_than_grid, 'finite_library_global_proof': finite_library_global_proof, 'continuous_global_proof': False, 'certificate_scope': certificate_scope}) / (clip_to_bounds(x0, bounds), {'success': False, 'fun': float('inf'), 'label': 'fallback_warm_start', 'nit': 0, 'method': 'fallback', 'candidates_evaluated': int(len(evaluated)), 'cem_used': False}) / refined / (run, spread)`
- 主なローカル代入名: `_`, `_FORK_COST_FN`, `axes`, `base_seeds`, `best`, `best_fun`, `bounds`, `candidate_fun`, `candidate_x`, `cem_mode_norm`, `cem_used`, `certificate_scope`, `clipped`, `completed`, `consensus_spread`, `declared_grid_values`, `default_seeds`, `deterministic_seed_candidates`, `deterministic_seed_nonfinite`, `discrete_global_proof`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 32、ループ 10、`try` 2

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `x0` に `np.asarray(x0, dtype=float)` の結果を代入する。
2. `bounds` に `list(bounds)` の結果を代入する。
3. `lo` に `np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)` の結果を代入する。
4. `hi` に `np.array([float(hi_i) for _, hi_i in bounds], dtype=float)` の結果を代入する。
5. `span` に `np.maximum(hi - lo, 1e-06)` の結果を代入する。
6. `rng` に `np.random.default_rng(int(rng_seed))` の結果を代入する。
7. `evaluated` に `[]` を代入する。
8. `seen` に `set()` を代入する。
9. 関数 `add_candidate` を定義する。
10. `base_seeds` に `list(structured_seeds or [])` の結果を代入する。
11. `seed_library_mode_norm` に `str(seed_library_mode or 'full').strip().lower()` の結果を代入する。
12. 条件 `seed_library_mode_norm not in {'full', 'realtime', 'minimal'}` を判定し、真なら内部処理を行う。
13. `seed_library_mode_norm` に `'full'` の結果を代入する。
14. `default_seeds` に `default_seed_library(x0, bounds)` の結果を代入する。
15. 条件 `seed_library_mode_norm == 'minimal'` を判定し、真なら内部処理を行う。
16. `default_seeds` に `default_seeds[:1]` の結果を代入する。
17. 上の条件が偽の場合:
18. 条件 `seed_library_mode_norm == 'realtime'` を判定し、真なら内部処理を行う。
19. `nominal` に `lo + 0.625 * span` の結果を代入する。
20. `default_seeds` に `[default_seeds[0], ('nominal_cruise', clip_to_bounds(nominal, bounds))]` の結果を代入する。
21. `default_seeds` を順に走査し、各要素を `(label, vec)` に入れて処理する。
22. `add_candidate(...)` を実行する。

23. `base_seeds` を順に走査し、各要素を `(label, vec)` に入れて処理する。
24. `add_candidate(...)` を実行する。
25. `finite_pool` に `[row for row in evaluated if np.isfinite(row['fun'])]` の結果を代入する。
26. 条件 `not finite_pool` を判定し、真なら内部処理を行う。
27. `(clip_to_bounds(x0, bounds), {'success': False, 'fun': float('inf'), 'label': 'fallback_warm_start', 'nit': 0, 'method': 'fallback', 'candidates_evaluated': int(len(evaluated)), 'cem_used': False})` を返す。
28. `finite_pool.sort(...)` を実行する。
29. `best` に `dict(finite_pool[0])` の結果を代入する。
30. 関数 `local_refine_rows` を定義する。
31. `refine_pool` に `finite_pool[:max(1, int(local_refine_topk))]` の結果を代入する。
32. `refined_rows` に `local_refine_rows(refine_pool)` の結果を代入する。
33. `refined_rows` を順に走査し、各要素を `row` に入れて処理する。
34. `evaluated.append(...)` を実行する。
35. 条件 `refined_rows` and `float(refined_rows[0]['fun']) < float(best['fun'])` を判定し、真なら内部処理を行う。
36. `best` に `dict(refined_rows[0])` の結果を代入する。
37. `cem_mode_norm` に `str(cem_mode or 'auto').strip().lower()` の結果を代入する。
38. 条件 `cem_mode_norm not in {'always', 'auto', 'never', 'shgo', 'certify'}` を判定し、真なら内部処理を行う。
39. `cem_mode_norm` に `'auto'` の結果を代入する。
40. `consensus_spread` に `float('nan')` の結果を代入する。
41. `cem_used` に `False` の結果を代入する。
42. `shgo_used` に `False` の結果を代入する。
43. `shgo_nfev` に `0` の結果を代入する。
44. `shgo_local_minima` に `0` の結果を代入する。
45. `discrete_global_proof` に `False` の結果を代入する。
46. `discrete_grid_candidates` に `0` の結果を代入する。
47. `discrete_grid_nonfinite` に `0` の結果を代入する。
48. `declared_grid_values` に `np.asarray(list(cert_grid_values or []), dtype=float)` の結果を代入する。
49. `declared_grid_values` に `declared_grid_values[np.isfinite(declared_grid_values)]` の結果を代入する。
50. 条件 `declared_grid_values.size` を判定し、真なら内部処理を行う。
51. `grid_values` に `np.unique(np.clip(declared_grid_values, lo[0], hi[0]))` の結果を代入する。
52. `discrete_grid_levels` に `int(len(grid_values))` の結果を代入する。
53. 上の条件が偽の場合:
54. `grid_values` に `np.asarray([], dtype=float)` の結果を代入する。
55. `discrete_grid_levels` に `max(0, int(cert_grid_levels))` の結果を代入する。

56. `deterministic_seed_candidates` に `int(len(evaluated))` の結果を代入する。
57. `deterministic_seed_nonfinite` に `int(sum((not np.isfinite(float(row.get('fun', float('inf'))))) for row in evaluated)))` の結果を代入する。
58. `discrete_grid_best_fun` に `float('inf')` の結果を代入する。
59. `discrete_grid_best_x` に `[]` を代入する。
60. 関数 `emit_progress` を定義する。
61. 条件 `cem_mode_norm == 'certify' and discrete_grid_levels >= 2 and (len(x0) > 0)` を判定し、真なら内部処理を行う。
62. `requested` に `int(discrete_grid_levels ** len(x0))` の結果を代入する。
63. 条件 `requested <= max(1, int(cert_max_evaluations))` を判定し、真なら内部処理を行う。
64. `emit_progress(...)` を実行する。
65. `axes` に `[grid_values.copy() if grid_values.size else np.linspace(lo_i, hi_i, discrete_grid_levels) for lo_i, hi_i in bounds]` の結果を代入する。
66. `grid_best` に `None` の結果を代入する。
67. `interval` に `max(1, int(cert_progress_interval))` の結果を代入する。
68. `workers` に `max(1, int(cert_workers))` の結果を代入する。
69. `executor_kind` に `'serial'` の結果を代入する。
70. 関数 `record_grid_candidate` を定義する。
71. `grid_candidates` に `[(idx, np.asarray(values, dtype=float)) for idx, values in enumerate(product(*axes))]` の結果を代入する。
72. 条件 `workers == 1` を判定し、真なら内部処理を行う。
73. `grid_candidates` を順に走査し、各要素を `(idx, candidate_x)` に入れて処理する。
74. 上の条件が偽の場合:
75. 条件 `os.name == 'posix'` を判定し、真なら内部処理を行う。
76. `with` 文で `executor_context` を管理しながら処理する。
77. 条件 `grid_best is not None and np.isfinite(float(grid_best['fun']))` を判定し、真なら内部処理を行う。
78. `discrete_grid_best_fun` に `float(grid_best['fun'])` の結果を代入する。
79. `discrete_grid_best_x` に `np.asarray(grid_best['x'], dtype=float).tolist()` の結果を代入する。
80. `evaluated.append(...)` を実行する。

代表コード断片

```
def hybrid_bounded_minimize(  
    cost_fn: Callable[[np.ndarray], float],  
    x0: np.ndarray,  
    bounds: Bounds,  
    *,  
    maxiter: int,  
    structured_seeds: Iterable[tuple[str, np.ndarray]] | None = None,  
    cem_enabled: bool = True,  
    cem_mode: str = "auto",  
    cem_generations: int = 4,
```



```

cem_population: int = 16,
cem_elite: int = 4,
local_refine_topk: int = 4,
seed_library_mode: str = "full",
rng_seed: int = 0,
shgo_samples: int = 256,
shgo_iters: int = 2,
cert_grid_levels: int = 0,
cert_grid_values: Sequence[float] | None = None,
cert_max_evaluations: int = 250_000,
cert_workers: int = 1,
progress_callback: Callable[[dict], None] | None = None,
cert_progress_interval: int = 25,
) -> tuple[np.ndarray, dict]:
    x0 = np.asarray(x0, dtype=float)
    bounds = list(bounds)
    lo = np.array([float(lo_i) for lo_i, _ in bounds], dtype=float)
    hi = np.array([float(hi_i) for _, hi_i in bounds], dtype=float)
    span = np.maximum(hi - lo, 1.0e-6)
    rng = np.random.default_rng(int(rng_seed))

    evaluated: list[dict] = []
    seen: set[tuple[float, ...]] = set()

    def add_candidate(label: str, vec: np.ndarray) -> None:
...

```

7.6 L91 関数 hybrid_bounded_minimize.add_candidate

- 行範囲: L91–L105
- 所属: hybrid_bounded_minimize
- 定義: add_candidate(label:str,vec:np.ndarray)->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: add, append, clip_to_bounds, finite_cost, round, str, tuple
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: clipped, key, value
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. clipped に clip_to_bounds(vec, bounds) の結果を代入する。
2. key に tuple(np.round(clipped, 6)) の結果を代入する。

3. 条件 `key in seen` を判定し、真なら内部処理を行う。
4. を返す。
5. `seen.add(...)` を実行する。
6. `value` に `finite_cost(cost_fn, clipped)` の結果を代入する。
7. `evaluated.append(...)` を実行する。

代表コード断片

```
def add_candidate(label: str, vec: np.ndarray) -> None:
    clipped = clip_to_bounds(vec, bounds)
    key = tuple(np.round(clipped, 6))
    if key in seen:
        return
    seen.add(key)
    value = finite_cost(cost_fn, clipped)
    evaluated.append(
        {
            "label": str(label),
            "x": clipped,
            "fun": value,
            "source": "seed",
        }
    )
```

7.7 L143 関数 `hybrid_bounded_minimize.local_refine_rows`

- 行範囲: L143–L165
- 所属: `hybrid_bounded_minimize`
- 定義: `local_refine_rows(pool: Sequence[dict]) -> list[dict]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `all`, `append`, `asarray`, `bool`, `clip_to_bounds`, `dict`, `finite_cost`, `float`, `getattr`, `hasattr`, `int`, `isfinite`
- 戻り値の要点: `refined / refined`
- 主なローカル代入名: `candidate_fun`, `candidate_x`, `refined`, `res`, `row`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

上から順の処理

1. `refined` に `[]` を代入する。
2. 条件 `maxiter <= 0` を判定し、真なら内部処理を行う。
3. `refined` を返す。
4. `pool` を順に走査し、各要素を `row` に入れて処理する。
5. `res` に `minimize(cost_fn, row['x'], method='L-BFGS-B', bounds=bounds, options=dict(maxiter=int(maxiter)))` の結果を代入する。
6. `candidate_x` に `row['x']` の結果を代入する。
7. 条件 `hasattr(res, 'x')` and `res.x is not None` and `(len(res.x) == len(x0))` and `np.all(np.isfinite(res.x))` を判定し、真なら内部処理を行う。
8. `candidate_x` に `clip_to_bounds(np.asarray(res.x, dtype=float), bounds)` の結果を代入する。
9. `candidate_fun` に `finite_cost(cost_fn, candidate_x)` の結果を代入する。
10. `refined.append(...)` を実行する。
11. `refined.sort(...)` を実行する。
12. `refined` を返す。

代表コード断片

```
def local_refine_rows(pool: Sequence[dict]) -> list[dict]:
    refined: list[dict] = []
    if maxiter <= 0:
        return refined
    for row in pool:
        res = minimize(cost_fn, row["x"], method="L-BFGS-B", bounds=bounds, options=dict(
            maxiter=int(maxiter)))
        candidate_x = row["x"]
        if hasattr(res, "x") and res.x is not None and len(res.x) == len(x0) and np.all(
            np.isfinite(res.x)):
            candidate_x = clip_to_bounds(np.asarray(res.x, dtype=float), bounds)
            candidate_fun = finite_cost(cost_fn, candidate_x)
            refined.append(
                {
                    "label": f"local_refine:{row['label']}",
                    "x": candidate_x,
                    "fun": candidate_fun,
                    "source": "local_refine",
                    "nit": int(getattr(res, "nit", 0) or 0),
                    "success": bool(getattr(res, "success", False)),
                    "message": str(getattr(res, "message", "")),
                }
            )
    refined.sort(key=lambda item: float(item["fun"]))
    return refined
```

7.8 L201 関数 `hybrid_bounded_minimize.emit_progress`

- 行範囲: L201–L208
- 所属: `hybrid_bounded_minimize`
- 定義: `emit_progress(payload:dict)->None`

- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `dict`, `progress_callback`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 1

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `progress_callback is None` を判定し、真なら内部処理を行う。
2. を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `progress_callback(...)` を実行する。
5. `Exception` を捕捉した場合:
6. を返す。

代表コード断片

```
def emit_progress(payload: dict) -> None:
    if progress_callback is None:
        return
    try:
        progress_callback(dict(payload))
    except Exception:
        # Progress telemetry must never change the optimizer result.
        return
```

7.9 L369 関数 `hybrid_bounded_minimize.should_run_cem_from_seed_consensus`

- 行範囲: L369–L380
- 所属: `hybrid_bounded_minimize`
- 定義: `should_run_cem_from_seed_consensus(rows: Sequence[dict]) -> tuple[bool, float]`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `asarray`, `bool`, `float`, `get`, `isfinite`, `len`, `max`, `maximum`, `min`, `std`, `vstack`
- 戻り値の要点: `(run, spread) / (False, 0.0)`

- 主なローカル代入名: `best_fun`, `gap`, `mat`, `next_fun`, `row`, `run`, `spread`, `top`, `usable`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `usable` に `[row for row in rows if np.isfinite(float(row.get('fun', float('inf'))))]` の結果を代入する。
2. 条件 `len(usable) < 2` を判定し、真なら内部処理を行う。
3. `(False, 0.0)` を返す。
4. `top` に `usable[:min(3, len(usable))]` の結果を代入する。
5. `mat` に `np.vstack([np.asarray(row['x'], dtype=float) for row in top])` の結果を代入する。
6. `spread` に `float(np.max(np.std(mat, axis=0) / np.maximum(span, 1e-06)))` の結果を代入する。
7. `best_fun` に `float(top[0]['fun'])` の結果を代入する。
8. `next_fun` に `float(top[1]['fun'])` の結果を代入する。
9. `gap` に `abs(next_fun - best_fun) / max(1.0, abs(best_fun))` の結果を代入する。
10. `run` に `bool(spread > 0.08 or gap > 0.1 or (not bool(top[0].get('success', True))))` の結果を代入する。
11. `(run, spread)` を返す。

代表コード断片

```
def should_run_cem_from_seed_consensus(rows: Sequence[dict]) -> tuple[bool, float]:
    usable = [row for row in rows if np.isfinite(float(row.get("fun", float("inf"))))]
    if len(usable) < 2:
        return False, 0.0
    top = usable[: min(3, len(usable))]
    mat = np.vstack([np.asarray(row["x"], dtype=float) for row in top])
    spread = float(np.max(np.std(mat, axis=0) / np.maximum(span, 1.0e-6)))
    best_fun = float(top[0]["fun"])
    next_fun = float(top[1]["fun"])
    gap = abs(next_fun - best_fun) / max(1.0, abs(best_fun))
    run = bool((spread > 0.08) or (gap > 0.10) or (not bool(top[0].get("success", True))))
    return run, spread
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/upper_solver.py` を **planner helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。